

LIBRERÍAS MODULARES EN STM32

DESARROLLO DE UN CONTROLADOR PARA EL MONITOR DE
CORRIENTE INA236

Autor:
Rodrigo CASTILLEJOS

Proyecto:
Control de Dispositivos Embebidos

May 31, 2026

RESUMEN

Este documento técnico expone de manera detallada el proceso metodológico para el diseño, desarrollo e integración de librerías de software modulares en lenguajes C y C++ destinadas a microcontroladores de la familia STM32. Para ilustrar estos conceptos de manera práctica, se presenta como caso de estudio la implementación de un controlador de bajo nivel para el sensor digital de corriente, tensión y potencia INA236 a través del bus de comunicación I2C utilizando la interfaz HAL (*Hardware Abstraction Layer*) de STMicroelectronics.

A lo largo de este informe se detalla la estructuración óptima de archivos de cabecera (.h) y archivos de código fuente (.c), las consideraciones físicas y matemáticas de la calibración interna del sensor basadas en las ecuaciones del fabricante, el manejo eficiente de variables de punto flotante en microcontroladores con núcleos ARM Cortex-M, y la configuración del entorno de compilación unificado CMake en Visual Studio Code. Este enfoque estructurado promueve la reutilización de código, facilita la depuración de hardware y garantiza una arquitectura de software embebido robusta y escalable.

TABLA DE CONTENIDOS

1	Fundamentos de Librerías en C para Sistemas Embebidos	4
2	Estructura Estándar de Archivos Header (.h) y Source (.c)	5
2.1	Estructura de un Archivo de Cabecera (.h)	5
2.2	Estructura de un Archivo de Código Fuente (.c)	5
3	Análisis Detallado del Archivo de Cabecera (INA236.h)	7
3.1	Constantes y Macros de Utilidad	7
3.2	Definición del Mapa de Registros	7
3.3	Máscaras y Posiciones de Bits	8
3.4	Tipos de Datos Enumerados (enum)	10
3.5	La Estructura de Control Principal (INA236_HandleTypeDef)	11
3.6	Prototipos de las Funciones del API	12
4	El Sensor de Monitoreo de Corriente y Potencia INA236	14
4.1	Especificaciones Técnicas Clave	14
4.2	Mapa de Registros Internos	14
4.3	Matemáticas de Calibración del Sensor	15
4.3.1	Cálculo del Current_LSB Mínimo	15
4.3.2	Cálculo del Registro SHUNT_CAL	15
4.3.3	Fórmulas de Conversión a Magnitudes Físicas en STM32	15
4.4	El Problema de Lecturas Nulas en Potencia ante Bajas Magnitudes	16
5	Análisis Detallado de las Funciones de la Librería INA236	17
5.1	Funciones de Comunicación de Bajo Nivel	17
5.1.1	Función INA236_WriteRegister	17
5.1.2	Función INA236_ReadRegister	18
5.2	Inicialización y Calibración Física del Sensor	19
5.2.1	Función INA236_Init	19
5.2.2	Función INA236_RoundCurrentLsb (Auxiliar Privada)	20
5.2.3	Función INA236_SetCalibration	21
5.2.4	Función INA236_ResetDevice	22
5.3	Funciones de Configuración de Periféricos	22
5.3.1	Función INA236_SetMode	22
5.3.2	Función INA236_SetShuntConvTime	23
5.3.3	Función INA236_SetBusConvTime	23
5.3.4	Función INA236_SetAverage	24
5.3.5	Función INA236_SetAdcRange	24
5.4	Funciones de Gestión de Alertas por Hardware	25
5.4.1	Configuración de Polaridad del Pin ALERT	25
5.4.2	Configuración de Alerta por Conversión Completa	26
5.4.3	Funciones de Selección de Límites de Alerta	26
5.4.4	Función INA236_SetAlertLimit	29
5.5	Funciones de Lectura de Diagnóstico e Identificación	30
5.5.1	Lectura de Banderas de Conversión y Desbordamiento	30
5.5.2	Lectura de Identificadores del Dispositivo	31
5.5.3	Lectura de Registros Internos de Configuración y Calibración	31
5.6	Funciones de Instrumentación y Medida Física	32
5.6.1	Lectura de Tensión de Shunt	32
5.6.2	Lectura de Tensión de Bus	32
5.6.3	Lectura de Corriente	33
5.6.4	Lectura de Potencia Consumida	33
6	Integración de la Librería en CMake y Soporte de Punto Flotante	34
6.1	Modificación del archivo CMakeLists.txt	34
6.2	Habilitación de Punto Flotante en la Toolchain (Newlib-nano)	34
	Anexo: Manipulación de Bits en Lenguaje C para Sistemas Embebidos	35

1 FUNDAMENTOS DE LIBRERÍAS EN C PARA SISTEMAS EMBEBIDOS

En el ámbito del diseño de firmware para sistemas embebidos, la modularidad y el encapsulamiento de funciones son indispensables para desarrollar código robusto, legible y reutilizable. Una **librería** en C se define como un conjunto de archivos independientes que empaquetan registros, estructuras de datos, macros y funciones diseñados para cumplir una tarea específica (como interactuar con un periférico de hardware o ejecutar un algoritmo de control), aislando dicha lógica del código principal de la aplicación (`main.c`).

Las ventajas clave del uso de librerías personalizadas en el desarrollo de firmware incluyen:

- **Abstracción de Hardware:** El programa principal interactúa con los sensores mediante llamadas a funciones legibles (por ejemplo, `INA236_GetCurrent_A()`) en lugar de manipular manualmente registros del bus I2C o realizar corrimientos de bits.
- **Reutilización de Código:** Permite importar el mismo controlador en múltiples proyectos con diferentes microcontroladores STM32 con mínimo esfuerzo de adaptación.
- **Aislamiento y Mantenimiento:** Si ocurre un error en la comunicación con el periférico o se modifica el diseño físico de la placa, los cambios se limitan al código interno de la librería, sin afectar la lógica superior de la aplicación.

El diseño estándar de una librería en C consta de la separación de su estructura en dos tipos de archivos fundamentales:

ARCHIVO DE CABECERA (.h): Representa la **interfaz pública**. Declara qué puede hacer la librería hacia el exterior. Contiene las definiciones de registros de hardware (direcciones físicas y máscaras de bits), tipos de datos de configuración (`enum` y `struct`), y los prototipos de las funciones que el usuario puede invocar desde otros archivos del proyecto.

ARCHIVO DE CÓDIGO FUENTE (.c): Representa la **implementación privada**. Define la lógica real de las funciones declaradas. Almacena funciones auxiliares y variables de configuración interna que están ocultas para el resto del programa, protegiendo el acceso indebido a la memoria del procesador.

El Proceso de Compilación y Enlazado

Durante el desarrollo del firmware, el preprocesador reemplaza recursivamente las directivas `#include` de los archivos `.h` en los fuentes `.c`. Posteriormente, el compilador procesa cada archivo `.c` por separado para generar archivos objeto intermedios (`.o`). Finalmente, el **Enlazador (Linker)** unifica estos objetos y resuelve las direcciones físicas de las funciones compartidas, estructurando el binario definitivo que se programará en la memoria Flash del microcontrolador.

2 ESTRUCTURA ESTÁNDAR DE ARCHIVOS HEADER (.h) Y SOURCE (.c)

Para garantizar la portabilidad y evitar conflictos de compilación en proyectos de STM32 bajo el ecosistema de CMake, se deben seguir estrictamente ciertos patrones de diseño al redactar archivos .h y .c.

2.1 Estructura de un Archivo de Cabecera (.h)

Un archivo de cabecera profesional debe contener los siguientes elementos ordenados de manera lógica:

1. **Guardas de Inclusión:** Evitan errores de doble declaración de tipos en el enlazador si el archivo es importado por múltiples dependencias. Se estructuran mediante las directivas `#ifndef`, `#define` y `#endif`.
2. **Inclusión de Dependencias:** Archivos del sistema como `stdint.h` (para tipos de enteros con tamaño fijo como `uint16_t`) y las librerías HAL específicas del microcontrolador (por ejemplo, `stm32f3xx_hal.h` o la cabecera general `main.h`).
3. **Definiciones de Constantes (Macros):** Direcciones físicas de registros I2C de lectura/escritura y máscaras para configurar registros de control a nivel de bit.
4. **Tipos de Datos Enumerados (enum):** Declaraciones de tipos con nombre que representan los valores admitidos de configuración (por ejemplo, promedios, tiempos de conversión y modos de operación).
5. **Estructura de Control (HandleTypeDef):** Una estructura en C que almacena las variables de estado e instancias físicas del periférico. Es la firma digital de la librería y vincula el controlador a un bus I2C real.
6. **Prototipos de Funciones Públicas:** Declaración formal de las funciones expuestas de la librería.

La figura 1 ilustra la anatomía estándar de un archivo de cabecera modular para sistemas embebidos.

```

/INA236.h
├── 1. Guardas de Inclusión (#ifndef INC_INA236_H_)
├── 2. Inclusión de dependencias (main.h,stdint.h)
├── 3. Constantes de registros (INA236_CONFIGURATION_REGISTER)
├── 4. Máscaras de control de bits (INA236_MODE_Mask)
├── 5. Enumeraciones de configuración (INA236_Avg_TypeDef)
├── 6. Estructura de control principal (INA236_HandleTypeDef)
├── 7. Prototipos de API pública (INA236_Init, INA236_GetCurrent_A)
└── 8. Cierre de Guardas (#endif)

```

Figura 1: Organización jerárquica del archivo de cabecera en C.

2.2 Estructura de un Archivo de Código Fuente (.c)

El archivo de implementación física organiza el código de la siguiente manera:

1. **Inclusión del Header Correspondiente:** Permite que el archivo compile conociendo todos los tipos y macros declarados en su cabecera (por ejemplo, `#include "INA236.h"`).
2. **Prototipos de Funciones Estáticas Privadas:** Declaraciones de funciones locales a las que solo se puede acceder dentro del mismo archivo fuente.

3. **Implementación de Funciones Auxiliares Privadas (`static`):** Funciones de soporte lógico interno.
4. **Implementación de Funciones del API Público:** Bloque con el desarrollo del código real para inicialización, lectura y control físico del sensor.

⚠ La Importancia del Modificador `static`

Al declarar una función o variable global como `static` en el archivo `.c`, se le indica al compilador que restrinja su visibilidad exclusivamente a ese módulo. Esto previene conflictos de nombres si otra librería utiliza un nombre idéntico, y permite que el optimizador del compilador genere llamadas a función en línea (*inline*) mucho más rápidas y eficientes en ensamblador.

3 ANÁLISIS DETALLADO DEL ARCHIVO DE CABECERA (ina236.h)

El archivo de cabecera INA236.h constituye la interfaz pública de la librería. En él se definen las constantes de dirección I2C, el mapa de registros físicos del sensor, las máscaras y posiciones de bits para configuraciones internas, los tipos de datos enumerados de control, la estructura principal de manejo del dispositivo y los prototipos de las funciones de la API.

3.1 Constantes y Macros de Utilidad

Al inicio del archivo se configuran los parámetros del bus I2C y una macro de ayuda para evaluación de bits:

```
1 #define INA236_DEFAULT_ADDRESS      0x40
2 #define INA236_TRIALS                5
3 #define CHECK_BIT(var,pos) ((var) & (1<<(pos)))
```

- INA236_DEFAULT_ADDRESS (0x40): Dirección I2C predeterminada de 7 bits del integrado cuando las entradas físicas de dirección Ao y A1 se conectan a la señal de GND.
- INA236_TRIALS (5): Número de reintentos máximos de consulta de presencia que el driver ejecutará en el bus I2C antes de declarar un error de desconexión analógica.
- CHECK_BIT(var, pos): Macro que evalúa de forma rápida el estado lógico de un bit específico en una variable de control, devolviendo un valor distinto de cero si el bit en la posición pos está en nivel alto (1):

$$\text{CHECK_BIT}(\text{var}, \text{pos}) = \text{var} \& (1 \ll \text{pos}) \quad (1)$$

3.2 Definición del Mapa de Registros

Las direcciones físicas de los registros del INA236 se declaran como macros de preprocesador para asociar nombres lógicos legibles con sus posiciones reales de memoria en el sensor:

```
1 /* INA236 Register Map Addresses */
2 #define INA236_CONFIGURATION_REGISTER 0x00 /* Configuration Register - Reset Value = 4127h*/
3 #define INA236_SHUNT_VOLTAGE_REGISTER 0x01 /* Shunt Voltage Register - Reset Value = 0000h*/
4 #define INA236_BUS_VOLTAGE_REGISTER 0x02 /* Bus Voltage Register - Reset Value = 0000h*/
5 #define INA236_POWER_REGISTER 0x03 /* Power Register - Reset Value = 0000h*/
6 #define INA236_CURRENT_REGISTER 0x04 /* Current Register - Reset Value = 0000h*/
7 #define INA236_CALIBRATION_REGISTER 0x05 /* Calibration Register - Reset Value = 0000h*/
8 #define INA236_MASK_ENABLE_REGISTER 0x06 /* Mask/Enable Register - Reset Value = 0000h*/
9 #define INA236_ALERT_LIMIT_REGISTER 0x07 /* Alert Limit Register - Reset Value = 0000h*/
10 #define MANUFACTURER_ID_REGISTER 0x3E /* Device ID Register - Reset Value = 5449h*/
11 #define DEVICE_ID_REGISTER 0x3F /* Manufacturer ID Register-Reset Value = A080h*/
```

- INA236_CONFIGURATION_REGISTER (0x00): Registro de 16 bits para programar resets, rangos, promedios y tiempos de conversión.
- INA236_SHUNT_VOLTAGE_REGISTER (0x01): Almacena la tensión diferencial en bornes del shunt.
- INA236_BUS_VOLTAGE_REGISTER (0x02): Almacena la tensión del bus común.
- INA236_POWER_REGISTER (0x03): Almacena el resultado del cálculo interno de potencia.
- INA236_CURRENT_REGISTER (0x04): Almacena el cálculo interno de corriente.
- INA236_CALIBRATION_REGISTER (0x05): Almacena el coeficiente de calibración de escala.

- INA236_MASK_ENABLE_REGISTER (0x06): Registro para configurar alertas e interrupciones.
- INA236_ALERT_LIMIT_REGISTER (0x07): Almacena el límite físico de comparación de la alerta.
- MANUFACTURER_ID_REGISTER (0x3E) y DEVICE_ID_REGISTER (0x3F): Registros de solo lectura que almacenan las firmas de silicio de fábrica.

3.3 Máscaras y Posiciones de Bits

Para realizar operaciones de bits seguras y eficientes de lectura-modificación-escritura en los registros de control, se implementa un diseño estándar basado en dos macros por cada campo de bits:

- **Posición (_Pos):** Define el bit menos significativo o bit inicial del campo de configuración en la palabra de 16 bits (por ejemplo, el campo de promedio AVG inicia en el bit 9: `#define INA236_AVG_Pos (9U)`).
- **Máscara (_Mask):** Define la máscara lógica de bits activos que cubre todo el campo. Se genera desplazando el valor máximo del campo por su posición (por ejemplo: `#define INA236_AVG_Mask (0x7U << INA236_AVG_Pos)`).

Este diseño se aplica en el registro de configuración para los campos de modo (MODE), tiempo de conversión del shunt (VSHCT), tiempo del bus (VBUSCT), promedio (AVG), rango del ADC (ADCRANGE) y reinicio por software (RST). De igual forma, en el registro de alertas MASK_ENABLE se definen las máscaras individuales de 1 bit para evaluar banderas como conmutación de alerta por conversión lista (CNVR), alerta por sobrepotencia (POL), caída de tensión en bus (BUL), sobretensión en bus (BOL), caída en shunt (SUL) o sobretensión en shunt (SOL), además de la polaridad de alerta (APOL) y la bandera de desbordamiento matemático (OVF).

```

1  /***** Bits definition for CONFIGURATION register *****/
2  #define INA236_MODE_Pos          (0U)
3  #define INA236_MODE_Mask        (0x7U << INA236_MODE_Pos)
4  #define INA236_MODE             INA236_MODE_Mask
5  #define INA236_MODE_0           (0x1U << INA236_MODE_Pos)
6  #define INA236_MODE_1           (0x2U << INA236_MODE_Pos)
7  #define INA236_MODE_2           (0x4U << INA236_MODE_Pos)
8
9  #define INA236_VSHCT_Pos         (3U)
10 #define INA236_VSHCT_Mask       (0x7U << INA236_VSHCT_Pos)
11 #define INA236_VSHCT           INA236_VSHCT_Mask
12 #define INA236_VSHCT_0          (0x1U << INA236_VSHCT_Pos)
13 #define INA236_VSHCT_1          (0x2U << INA236_VSHCT_Pos)
14 #define INA236_VSHCT_2          (0x4U << INA236_VSHCT_Pos)
15
16 #define INA236_VBUSCT_Pos        (6U)
17 #define INA236_VBUSCT_Mask      (0x7U << INA236_VBUSCT_Pos)
18 #define INA236_VBUSCT           INA236_VBUSCT_Mask
19 #define INA236_VBUSCT_0         (0x1U << INA236_VBUSCT_Pos)
20 #define INA236_VBUSCT_1         (0x2U << INA236_VBUSCT_Pos)
21 #define INA236_VBUSCT_2         (0x4U << INA236_VBUSCT_Pos)
22
23 #define INA236_AVG_Pos           (9U)
24 #define INA236_AVG_Mask         (0x7U << INA236_AVG_Pos)
25 #define INA236_AVG              INA236_AVG_Mask
26 #define INA236_AVG_0            (0x1U << INA236_AVG_Pos)
27 #define INA236_AVG_1            (0x2U << INA236_AVG_Pos)
28 #define INA236_AVG_2            (0x4U << INA236_AVG_Pos)
29
30 #define INA236_ADCRANGE_Pos      (12U)

```



```

31 #define INA236_ADCRANGE_Mask      (0x1U << INA236_ADCRANGE_Pos)
32 #define INA236_ADCRANGE          INA236_ADCRANGE_Mask
33
34 #define INA236_RST_Pos            (15U)
35 #define INA236_RST_Mask          (0x1U << INA236_RST_Pos)
36 #define INA236_RST                INA236_RST_Mask
37
38 /***** Bits definition for MASK/ENABLE register *****/
39 #define INA236_LEN_Pos            (0U)
40 #define INA236_LEN_Mask          (0x1U << INA236_LEN_Pos)
41 #define INA236_LEN                INA236_LEN_Mask
42
43 #define INA236_AP0L_Pos           (1U)
44 #define INA236_AP0L_Mask         (0x1U << INA236_AP0L_Pos)
45 #define INA236_AP0L              INA236_AP0L_Mask
46
47 #define INA236_OVF_Pos           (2U)
48 #define INA236_OVF_Mask          (0x1U << INA236_OVF_Pos)
49 #define INA236_OVF                INA236_OVF_Mask
50
51 #define INA236_CVRF_Pos          (3U)
52 #define INA236_CVRF_Mask         (0x1U << INA236_CVRF_Pos)
53 #define INA236_CVRF              INA236_CVRF_Mask
54
55 #define INA236_AFF_Pos           (4U)
56 #define INA236_AFF_Mask          (0x1U << INA236_AFF_Pos)
57 #define INA236_AFF                INA236_AFF_Mask
58
59 #define INA236_MEMERROR_Pos      (5U)
60 #define INA236_MEMERROR_Mask     (0x1U << INA236_MEMERROR_Pos)
61 #define INA236_MEMERROR          INA236_MEMERROR_Mask
62
63 #define INA236_CNVR_Pos          (10U)
64 #define INA236_CNVR_Mask         (0x1U << INA236_CNVR_Pos)
65 #define INA236_CNVR              INA236_CNVR_Mask
66
67 #define INA236_POL_Pos           (11U)
68 #define INA236_POL_Mask          (0x1U << INA236_POL_Pos)
69 #define INA236_POL                INA236_POL_Mask
70
71 #define INA236_BUL_Pos           (12U)
72 #define INA236_BUL_Mask          (0x1U << INA236_BUL_Pos)
73 #define INA236_BUL                INA236_BUL_Mask
74
75 #define INA236_B0L_Pos           (13U)
76 #define INA236_B0L_Mask          (0x1U << INA236_B0L_Pos)
77 #define INA236_B0L                INA236_B0L_Mask
78
79 #define INA236_SUL_Pos           (14U)
80 #define INA236_SUL_Mask          (0x1U << INA236_SUL_Pos)
81 #define INA236_SUL                INA236_SUL_Mask
82
83 #define INA236_S0L_Pos           (15U)
84 #define INA236_S0L_Mask          (0x1U << INA236_S0L_Pos)
85 #define INA236_S0L                INA236_S0L_Mask

```

3.4 Tipos de Datos Enumerados (enum)

Se definen tipos enumerados fuertemente tipados para restringir los parámetros que recibe la API, garantizando que el compilador de C verifique la validez de los argumentos:

ina236_mode_typedef: Define los modos del ADC del chip (Shutdown, mediciones disparadas de bus/shunt, o mediciones continuas).

ina236_convtime_typedef: Tiempos de integración analógica permitidos para el ADC (rango de 140 μ s a 8244 μ s).

ina236_avg_typedef: Opciones de promediado digital de hardware (de 1 a 1024 muestras).

ina236_adcrange_typedef: Configura la ganancia del ADC analógico frontal (± 81.92 mV o ± 20.48 mV).

ina236_alertpol_typedef: Configura si el pin físico ALERT conmuta a nivel lógico bajo (ACTIVE_LOW) o alto (ACTIVE_HIGH).

ina236_alerttype_typedef: Define el tipo de alerta activa que el controlador usará para determinar el tipo de escalado del límite físico.

```

1  typedef enum
2  {
3      INA236_SHUTDOWN                = 0x0U,
4      INA236_SHUNT_VOLTAGE_ONE_SHOT  = 0x1U,
5      INA236_BUS_VOLTAGE_ONE_SHOT    = 0x2U,
6      INA236_SHUNT_BUS_VOLTAGE_ONE_SHOT = 0x3U,
7      INA236_CONTINUOUS_SHUNT_VOLTAGE = 0x5U,
8      INA236_CONTINUOUS_BUS_VOLTAGE   = 0x6U,
9      INA236_CONTINUOUS_SHUNT_BUS_VOLTAGE = 0x7U
10 } INA236_Mode_TypeDef;
11
12 typedef enum
13 {
14     INA236_140_US                = 0x0U,
15     INA236_204_US                = 0x1U,
16     INA236_332_US                = 0x2U,
17     INA236_588_US                = 0x3U,
18     INA236_1100_US               = 0x4U,
19     INA236_2116_US               = 0x5U,
20     INA236_4156_US               = 0x6U,
21     INA236_8244_US               = 0x7U
22 } INA236_ConvTime_TypeDef;
23
24 typedef enum
25 {
26     INA236_1_SAMPLE               = 0x0U,
27     INA236_4_SAMPLES              = 0x1U,
28     INA236_16_SAMPLES             = 0x2U,
29     INA236_64_SAMPLES             = 0x3U,
30     INA236_128_SAMPLES            = 0x4U,
31     INA236_256_SAMPLES            = 0x5U,
32     INA236_512_SAMPLES            = 0x6U,
33     INA236_1014_SAMPLES           = 0x7U
34 } INA236_Avg_TypeDef;
35
36 typedef enum
37 {
38     INA236_ADC_RANGE_81_92_MV     = 0x0U,
39     INA236_ADC_RANGE20_48_MV      = 0x1U

```

```

40 } INA236_AdcRange_TypeDef;
41
42 typedef enum
43 {
44     INA236_ACTIVE_LOW           = 0x0U,
45     INA236_ACTIVE_HIGH         = 0x1U
46 } INA236_AlertPol_TypeDef;
47
48 typedef enum
49 {
50     INA236_CONV_READY_ALERT_DISABLE = 0x0U,
51     INA236_CONV_READY_ALERT_ENABLE = 0x1U
52 } INA236_ConvRdy_TypeDef;
53
54 typedef enum
55 {
56     INA236_POWER_LIMIT_ALERT_DISABLE = 0x0U,
57     INA236_POWER_LIMIT_ALERT_ENABLE = 0x1U
58 } INA236_PowerAlert_TypeDef;
59
60 typedef enum
61 {
62     INA236_BUS_UNDER_LIMIT_ALERT_DISABLE = 0x0U,
63     INA236_BUS_UNDER_LIMIT_ALERT_ENABLE = 0x1U
64 } INA236_BusUnderLimitAlert_TypeDef;
65
66 typedef enum
67 {
68     INA236_BUS_OVER_LIMIT_ALERT_DISABLE = 0x0U,
69     INA236_BUS_OVER_LIMIT_ALERT_ENABLE = 0x1U
70 } INA236_BusOverLimitAlert_TypeDef;
71
72 typedef enum
73 {
74     INA236_SHUNT_UNDER_LIMIT_ALERT_DISABLE = 0x0U,
75     INA236_SHUNT_UNDER_LIMIT_ALERT_ENABLE = 0x1U
76 } INA236_ShuntUnderLimitAlert_TypeDef;
77
78 typedef enum
79 {
80     INA236_SHUNT_OVER_LIMIT_ALERT_DISABLE = 0x0U,
81     INA236_SHUNT_OVER_LIMIT_ALERT_ENABLE = 0x1U
82 } INA236_ShuntOverLimitAlert_TypeDef;
83
84 typedef enum
85 {
86     INA236_SOL_ALERT = 0x1U,
87     INA236_SUL_ALERT = 0x2U,
88     INA236_BOL_ALERT = 0x3U,
89     INA236_BUL_ALERT = 0x4U,
90     INA236_POL_ALERT = 0x5U
91 } INA236_AlertType_TypeDef;

```

3.5 La Estructura de Control Principal (INA236_HandleTypeDef)

Es la estructura de datos que almacena toda la información del dispositivo en el software y vincula el driver con la instancia de hardware del microcontrolador. Cada campo tiene un propósito específico:

hi2c: Puntero a la estructura de la HAL del STM32 que gestiona el periférico físico I2C (por ejemplo, I2C_HandleTypeDef).

_devaddress: Almacena la dirección física configurada del sensor en el bus I2C.

_adcrange: Bandera lógica que registra si el ADC está configurado en rango bajo (1 para ± 20.48 mV) o alto (0 para ± 81.92 mV).

_resolution: Almacena la resolución física en Volts/LSB de tensión diferencial calculada.

_shuntresistor: Almacena el valor óhmico de la resistencia shunt de sensado soldado en la placa física.

_maximumcurrent: Almacena el valor límite de corriente de diseño de la aplicación.

_shuntadcrange: Almacena el rango analógico del ADC en Volts (0.08192 V o 0.02048 V).

_currentlsbmin: Guarda el cálculo del paso de corriente mínimo de hardware.

_currentlsb: Almacena el LSB de corriente redondeado final que sirve como factor multiplicador para las conversiones físicas de corriente y potencia.

_alerttype: Almacena el tipo de alerta activa de tipo INA236_AlertType_TypeDef.

```

1 typedef struct
2 {
3     I2C_HandleTypeDef      *hi2c;
4     uint8_t                _devAddress;
5     _Bool                  _adcRange;
6     double                 _resolution;
7     double                 _shuntResistor;
8     double                 _maximumCurrent;
9     double                 _shuntAdcRange;
10    double                  _currentLsbMin;
11    double                  _currentLsb;
12    INA236_AlertType_TypeDef _alertType;
13 } INA236_HandleTypeDef;

```

3.6 Prototipos de las Funciones del API

El archivo finaliza listando las declaraciones formales de las funciones públicas de la librería. Estas funciones están agrupadas lógicamente para cubrir las necesidades de:

1. **Capa de Comunicación Directa:** Funciones de bajo nivel para lectura/escritura física de registros I2C de 16 bits.
2. **Inicialización y Puesta en Marcha:** Configuración lógica de la estructura y autocalibración matemática del shunt.
3. **Configuración de Parámetros:** Ajustes dinámicos de modos de ADC, tiempos de conversión, promedio y rangos.
4. **Gestión de Alertas de Seguridad:** Control de polaridad, habilitación del pin y programación de umbrales máximos/mínimos.
5. **Instrumentación y Diagnóstico:** Lecturas de magnitudes físicas de tensión de shunt, tensión de bus, corriente y potencia, además de verificación de banderas de datos listos y desbordamientos.

```

1  /* Prototypes for low-level register interface */
2  HAL_StatusTypeDef INA236_WriteRegister(INA236_HandleTypeDef *ina236, uint8_t registerAddress,
    uint16_t value);
3  uint16_t INA236_ReadRegister(INA236_HandleTypeDef *ina236, uint8_t registerAddress);
4
5  /* Initialization & Calibration */
6  HAL_StatusTypeDef INA236_Init(INA236_HandleTypeDef *ina236, I2C_HandleTypeDef *i2c, uint8_t
    devAddress);
7  void INA236_SetCalibration(INA236_HandleTypeDef *ina236, double rShuntValue, int maxCurrent)
    ;
8  void INA236_ResetDevice(INA236_HandleTypeDef *ina236);
9
10 /* Configuration Setters & Getters */
11 void INA236_SetMode(INA236_HandleTypeDef *ina236, INA236_Mode_TypeDef mode);
12 void INA236_SetShuntConvTime(INA236_HandleTypeDef *ina236, INA236_ConvTime_TypeDef convTime)
    ;
13 void INA236_SetBusConvTime(INA236_HandleTypeDef *ina236, INA236_ConvTime_TypeDef convTime);
14 void INA236_SetAverage(INA236_HandleTypeDef *ina236, INA236_Avg_TypeDef avg);
15 void INA236_SetAdcRange(INA236_HandleTypeDef *ina236, INA236_AdcRange_TypeDef range);
16
17 /* Alert Management */
18 void INA236_SetAlertPolarity(INA236_HandleTypeDef *ina236, INA236_AlertPol_TypeDef polarity)
    ;
19 INA236_AlertPol_TypeDef INA236_GetAlertPolarity(INA236_HandleTypeDef *ina236);
20 void INA236_SetAlertPin(INA236_HandleTypeDef *ina236, INA236_ConvRdy_TypeDef cnvr);
21 INA236_ConvRdy_TypeDef INA236_GetAlertPin(INA236_HandleTypeDef *ina236);
22 void INA236_SetPowerOverLimit(INA236_HandleTypeDef *ina236, INA236_PowerAlert_TypeDef
    powerAlert);
23 void INA236_SetBusUnderLimit(INA236_HandleTypeDef *ina236, INA236_BusUnderLimitAlert_TypeDef
    bulAlert);
24 void INA236_SetBusOverLimit(INA236_HandleTypeDef *ina236, INA236_BusOverLimitAlert_TypeDef
    bolAlert);
25 void INA236_SetShuntUnderLimit(INA236_HandleTypeDef *ina236,
    INA236_ShuntUnderLimitAlert_TypeDef sulAlert);
26 void INA236_SetShuntOverLimit(INA236_HandleTypeDef *ina236,
    INA236_ShuntOverLimitAlert_TypeDef solAlert);
27 void INA236_SetAlertLimit(INA236_HandleTypeDef *ina236, int16_t alertLimit);
28
29 /* Diagnostic Readings */
30 _Bool INA236_IsConversionReady(INA236_HandleTypeDef *ina236);
31 uint8_t INA236_IsMathOverflowReady(INA236_HandleTypeDef *ina236);
32 uint16_t INA236_GetManufacturerID(INA236_HandleTypeDef *ina236);
33 uint16_t INA236_GetDeviceID(INA236_HandleTypeDef *ina236);
34 uint16_t INA236_GetConfiguration(INA236_HandleTypeDef *ina236);
35 uint16_t INA236_GetCalibration(INA236_HandleTypeDef *ina236);
36 uint16_t INA236_GetMaskEnable(INA236_HandleTypeDef *ina236);
37
38 /* Measurement Readings */
39 double INA236_GetShuntVoltage_mV(INA236_HandleTypeDef *ina236);
40 double INA236_GetBusVoltage(INA236_HandleTypeDef *ina236);
41 double INA236_GetCurrent_A(INA236_HandleTypeDef *ina236);
42 double INA236_GetPower_W(INA236_HandleTypeDef *ina236);

```

4 EL SENSOR DE MONITOREO DE CORRIENTE Y POTENCIA INA236

El **INA236** es un monitor digital de corriente bidireccional y tensión de bus de 16 bits de resolución, equipado con una interfaz compatible con buses I2C y SMBus. Es ampliamente utilizado en el diseño de fuentes de alimentación inteligentes, protección de sobrecorriente para motores y telemetría de energía en tarjetas de desarrollo embebidas.

4.1 Especificaciones Técnicas Clave

- **Tensión de Alimentación:** Opera en un rango de 1.7 V a 5.5 V.
- **Rango del Modo Común del Bus (VBUS):** Mide tensiones en la carga desde 0 V hasta 48 V de forma independiente de la alimentación.
- **Resolución del ADC Interno:** Conversor analógico a digital de 16 bits de ultra bajo ruido.
- **Rangos de Medición de Shunt Seleccionables:** Rango de entrada de ± 81.92 mV (para grandes corrientes) y ± 20.48 mV (para optimizar la resolución en corrientes pequeñas).
- **Parámetros de Calibración Interna:** Cuenta con un multiplicador matemático por hardware que calcula y devuelve de forma directa la lectura de corriente en Amperes e incluso la potencia del sistema en Watts tras configurarse su registro de calibración.
- **Entrada de Alerta Configurable:** Pin físico ALERT de drenador abierto para eventos de sobre/subtensión de bus, sobre/subcorriente o desbordamiento matemático.

4.2 Mapa de Registros Internos

El **INA236** cuenta con un mapa de registros de 16 bits de ancho accesibles a través de su interfaz I2C direccionable. La tabla 1 describe la organización física del chip.

Nombre del Registro	Dirección I2C	Acceso	Descripción Técnica
CONFIGURATION	0x00	R/W	Configuración del modo de conversión, tiempos de muestreo y rango del ADC.
SHUNT_VOLTAGE	0x01	R	Lectura cruda de la tensión diferencial en la resistencia shunt.
BUS_VOLTAGE	0x02	R	Lectura cruda de la tensión común del bus de alimentación externa.
POWER	0x03	R	Lectura cruda del cálculo interno de potencia en Watts.
CURRENT	0x04	R	Lectura cruda del cálculo de corriente en Amperes.
CALIBRATION	0x05	R/W	Registro del coeficiente matemático de calibración física del shunt.
MASK_ENABLE	0x06	R/W	Configuración de banderas de alertas físicas y bits de diagnóstico.
ALERT_LIMIT	0x07	R/W	Límite físico para la activación del pin ALERT.
MANUFACTURER_ID	0x3E	R	Firma de silicio del fabricante (Retorna 0x5449).
DEVICE_ID	0x3F	R	Firma de silicio del dispositivo (Retorna 0xA080).

Tabla 1: Mapa de registros de control e interfaz de hardware del **INA236**.

4.3 Matemáticas de Calibración del Sensor

Para que el procesador interno del INA236 calcule directamente los Amperes y los Watts sin cargar de trabajo al CPU del microcontrolador STM32, se debe programar el registro CALIBRATION (SHUNT_CAL) según las siguientes directrices matemáticas del fabricante:

4.3.1 Cálculo del Current_LSB Mínimo

Primero, se determina la resolución de paso de corriente más pequeña que se puede medir físicamente con el convertidor de 16 bits (donde $2^{15} = 32768$ valores positivos posibles en modo bipolar):

$$\text{Current_LSB}_{\min} = \frac{I_{\max_esperada}}{32768} \quad (2)$$

Una vez conocido el límite físico absoluto, se selecciona un valor redondeado de Current_LSB que sea un número entero o decimal "amigable" y limpio en pasos de 1, 2 o 5 veces una potencia de 10 (por ejemplo, 10 μA , 20 μA o 50 μA), garantizando que cumpla con la restricción de diseño:

$$\text{Current_LSB}_{\min} \leq \text{Current_LSB} < 2.5 \times \text{Current_LSB}_{\min} \quad (3)$$

4.3.2 Cálculo del Registro SHUNT_CAL

A partir del Current_LSB seleccionado y del valor resistivo de la resistencia del shunt (R_{shunt} en Ohms), se calcula el coeficiente entero a escribir en el registro de calibración del chip utilizando la constante de escala fija del fabricante (0.00512):

$$\text{SHUNT_CAL} = \frac{0.00512}{\text{Current_LSB} \times R_{\text{shunt}}} \quad (4)$$

⚠ Escalado por Rango de Tensión del ADC

El valor base de la ecuación anterior asume una configuración de rango analógico de entrada de $\pm 81.92 \text{ mV}$ (ADCRANGE = 0). Si se activa el modo de alta resolución de $\pm 20.48 \text{ mV}$ (ADCRANGE = 1), el valor obtenido en la fórmula del registro SHUNT_CAL **debe dividirse obligatoriamente entre 4** antes de programarse en el sensor.

4.3.3 Fórmulas de Conversión a Magnitudes Físicas en STM32

Una vez que el registro de calibración ha sido programado con éxito, los valores obtenidos de los registros se convierten a unidades del Sistema Internacional mediante las siguientes relaciones:

- **Tensión del Shunt (V_{shunt}):** Multiplica la lectura directa en dos de dos de dos de dos complementos por la resolución del ADC.

$$V_{\text{shunt}} [\text{mV}] = \text{Raw_Shunt} \times \text{Resolución (V)} \times 1000.0 \quad (5)$$

Donde la resolución es de 2.5 $\mu\text{V}/\text{LSB}$ para el rango de 81.92 mV, y de 0.625 $\mu\text{V}/\text{LSB}$ para el rango de 20.48 mV.

- **Tensión de Bus (V_{bus}):** Tiene un paso de tensión de entrada fijo de 1.6 mV/LSB (0.0016 V):

$$V_{\text{bus}} [\text{V}] = \text{Raw_Bus} \times 0.0016 \quad (6)$$

- **Corriente (I):** Multiplica directamente el registro firmado por el LSB físico configurado:

$$I [\text{A}] = \text{Raw_Current} \times \text{Current_LSB} \quad (7)$$

- **Potencia (P):** La potencia se calcula en base a la corriente y la tensión, y está acoplada al LSB por una constante de escala de 32 veces el LSB de corriente:

$$P [\text{W}] = \text{Raw_Power} \times (32.0 \times \text{Current_LSB}) \quad (8)$$

4.4 El Problema de Lecturas Nulas en Potencia ante Bajas Magnitudes

Un comportamiento reportado de forma común en el diseño de controladores para el INA236 es que, ante voltajes de bus o corrientes de carga muy pequeñas (por ejemplo, corriente de 11.7 mA y caída de tensión del shunt de apenas unos microvoltios), el registro físico de potencia `POWER` lee un valor absoluto de cero, a pesar de que el registro de corriente sí reporta lecturas válidas. Esto se debe al diseño interno de cálculo matemático de punto fijo del chip. El coprocesador interno del INA236 realiza la multiplicación directa de los registros crudos de corriente y tensión, aplicando una escala rígida en enteros:

$$\text{POWER}_{\text{raw}} = \frac{\text{CURRENT}_{\text{raw}} \times \text{VBUS}_{\text{raw}}}{20000} \quad (9)$$

Si el producto entre $\text{CURRENT}_{\text{raw}}$ y VBUS_{raw} es menor a 20000, la división entera trunca el resultado a 0. Para solventar esta limitación física en silicio y medir con alta fidelidad, existen dos alternativas de diseño:

1. **Reducir el Current_LSB:** Configurar un LSB de corriente más pequeño (por ejemplo, bajando de 1 mA a 100 μA), lo cual incrementará el valor del registro entero $\text{CURRENT}_{\text{raw}}$ para la misma corriente física, evitando el truncamiento a cero.
2. **Cálculo de Potencia por Software:** Si la corriente y la tensión del bus son extremadamente bajas y no se desea modificar la escala de hardware, se puede implementar el cálculo de la potencia en punto flotante directamente en el procesador del STM32 mediante software:

$$P [\text{W}] = I [\text{A}] \times V_{\text{bus}} [\text{V}] \quad (10)$$

5 ANÁLISIS DETALLADO DE LAS FUNCIONES DE LA LIBRERÍA INA236

En esta sección se detalla de forma exhaustiva el funcionamiento interno de cada una de las funciones que componen la API del driver del sensor INA236. Para cada función se especifica su propósito de hardware o software, el análisis de sus parámetros de entrada y salida, las operaciones a nivel de bit aplicadas y el comportamiento esperado del bus I2C.

5.1 Funciones de Comunicación de Bajo Nivel

Estas funciones implementan la interfaz directa con la capa física del protocolo I2C. Su principal tarea es realizar la traducción de formato entre el protocolo de bus Big-Endian (donde se transmite el byte más significativo primero en la red) y la arquitectura interna Little-Endian del núcleo ARM Cortex-M del microcontrolador STM32.

5.1.1 Función `INA236_WriteRegister`

Escribe un valor de control o configuración de 16 bits en una dirección física de registro en el silicio del sensor.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura de control de tipo `INA236_HandleTypeDef`. Utiliza el miembro `hi2c` para obtener el manejador físico del bus I2C de la HAL, y el miembro `_devAddress` para conocer la dirección esclava de 7 bits del chip.

registeraddress: Dirección de 8 bits del registro destino en el INA236 (por ejemplo, `0x00` para configuración o `0x05` para calibración).

value: Entero sin signo de 16 bits que contiene los datos que se grabarán en el registro seleccionado.

- **Retorno:** Código de estado de tipo `HAL_StatusTypeDef`. Devuelve `HAL_OK` si la escritura física se concreta con éxito o `HAL_ERROR` si el sensor no responde en el bus.

- **Funcionamiento Interno:** La función realiza una conversión de formato dividiendo el dato de 16 bits en dos bytes separados:

$$\text{address}[0] = (\text{value} \gg 8) \& 0xFF \quad (\text{Byte de peso alto / MSB}) \quad (11)$$

$$\text{address}[1] = (\text{value} \gg 0) \& 0xFF \quad (\text{Byte de peso bajo / LSB}) \quad (12)$$

Posteriormente, verifica la presencia física del dispositivo en la dirección esclava del bus I2C mediante una llamada de preparación `HAL_I2C_IsDeviceReady`, enviando la dirección de 7 bits desplazada un bit a la izquierda (`devAddress << 1`) para adecuar el formato de direccionamiento de 8 bits de la HAL de ST. Si el chip responde antes del tiempo de expiración (*Timeout*), se ejecuta la función de escritura directa de memoria de la HAL `HAL_I2C_Mem_Write` en modo de tamaño de dirección de registro de 8 bits (`I2C_MEMADD_SIZE_8BIT`) y tamaño de transmisión de búfer de 16 bits (`I2C_MEMADD_SIZE_16BIT`).

```

1  HAL_StatusTypeDef INA236_WriteRegister(INA236_HandleTypeDef *ina236, uint8_t registerAddress,
    uint16_t value)
2  {
3      uint8_t address[2];
4      HAL_StatusTypeDef isDeviceReady;
5      // Split the 16-bit value into two bytes (MSB first)
6      address[0] = (value >> 8) & 0xFF;
7      address[1] = (value >> 0) & 0xFF;
8
9      // Check if the device is ready on the I2C bus
10     isDeviceReady = HAL_I2C_IsDeviceReady(ina236->hi2c, (ina236->_devAddress) << 1,
        INA236_TRIALS, HAL_MAX_DELAY);

```

```

11  if (isDeviceReady == HAL_OK)
12  {
13      // Write the 16-bit register to the device
14      if (HAL_I2C_Mem_Write(ina236->hi2c, (ina236->_devAddress) << 1, registerAddress,
15                          I2C_MEMADD_SIZE_8BIT, (uint8_t*)address, I2C_MEMADD_SIZE_16BIT, HAL_MAX_DELAY) == HAL_OK
16      )
17      {
18          return HAL_OK;
19      }
20  }

```

5.1.2 Función INA236_ReadRegister

Lee los datos de 16 bits contenidos en un registro específico del mapa de memoria del INA236.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura INA236_HandleTypeDef para referenciar el bus I2C y la dirección del sensor.

registeraddress: Dirección física de 8 bits del registro al que se desea acceder para su lectura (por ejemplo, 0x01 para voltaje de shunt o 0x04 para corriente).

- **Retorno:** Entero de 16 bits sin signo con los datos leídos del registro. En caso de que ocurra una falla física de comunicación I2C o el dispositivo no esté presente en el bus, la función retorna el valor límite de control 0xFFFF.

- **Funcionamiento Interno:** La función valida la presencia del chip esclavo en el bus I2C mediante la directiva HAL_I2C_IsDeviceReady (desplazando la dirección _devAddress << 1). Si el estado de la comunicación es correcto, invoca a la función de lectura de memoria HAL_I2C_Mem_Read. Esta lectura extrae los datos de 16 bits del sensor recibidos en un búfer temporal de dos bytes de tamaño en formato Big-Endian. Finalmente, realiza una operación de desplazamiento y suma lógica OR para unir los bytes individuales en un entero nativo Little-Endian de 16 bits:

$$\text{Retorno} = (\text{registerResponse}[0] \ll 8) | \text{registerResponse}[1] \quad (13)$$

```

1  uint16_t INA236_ReadRegister(INA236_HandleTypeDef *ina236, uint8_t registerAddress)
2  {
3      uint8_t registerResponse[2] = {0};
4      HAL_StatusTypeDef isDeviceReady;
5
6      // Check if the device is ready on the I2C bus
7      isDeviceReady = HAL_I2C_IsDeviceReady(ina236->hi2c, (ina236->_devAddress) << 1,
8      INA236_TRIALS, HAL_MAX_DELAY);
9
10     if (isDeviceReady == HAL_OK)
11     {
12         // Read the 2-byte register data from the device
13         if (HAL_I2C_Mem_Read(ina236->hi2c, (ina236->_devAddress) << 1, registerAddress,
14                             I2C_MEMADD_SIZE_8BIT, registerResponse, sizeof(registerResponse), HAL_MAX_DELAY) ==
15         HAL_OK)
16         {
17             // Combine MSB and LSB to return the 16-bit value
18             return (uint16_t)((registerResponse[0] << 8) | registerResponse[1]);
19         }
20     }
21     // Returns a maximum control value (0xFFFF) indicating a communication error

```

```

19     return 0xFFFF;
20 }

```

5.2 Inicialización y Calibración Física del Sensor

Estas funciones configuran las variables lógicas iniciales del firmware, prueban el canal de hardware físico del bus de datos y cargan las constantes matemáticas del shunt y la corriente máxima en los registros de escala analógica.

5.2.1 Función `INA236_Init`

Vincula el hardware físico del microcontrolador al controlador de software embebido y establece la parametrización de funcionamiento por defecto.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura de manejo `INA236_HandleTypeDef` que se inicializará.

i2c: Puntero al manejador físico de I2C del STM32 (por ejemplo, `&hi2c1` o `&hi2c2`) generado previamente por CubeMX.

devaddress: Dirección base I2C del sensor en la tarjeta física de desarrollo (usualmente `0x40`).

- **Retorno:** Código de estado `HAL_StatusTypeDef` (`HAL_OK` o `HAL_ERROR`).

- **Funcionamiento Interno:** La función asigna los parámetros físicos locales a la estructura del driver para su uso en llamadas posteriores de la API. Inmediatamente ejecuta la comprobación de bus `HAL_I2C_IsDeviceReady`. Si el chip responde, procede a inicializar los registros de control interno del INA236 llamando a sus funciones de configuración secundaria con los siguientes parámetros predefinidos:

- **Modo de Operación:** Continuo para mediciones de tensión de shunt y bus (`INA236_CONTINUOUS_SHUNT_BUS_VOLTAGE`).
- **Tiempo de Conversión Shunt y Bus:** Establecido en 1.1 ms (`INA236_1100_US`), una velocidad estándar que ofrece una excelente filtración ante ruidos eléctricos transitorios.
- **Promedio de Muestreo:** Configurado en 64 muestras promedio (`INA236_64_SAMPLES`), reduciendo de forma significativa el ruido en las lecturas de salida a costa de un retraso de actualización en milisegundos.
- **Rango del ADC:** Ajustado en su valor máximo de ± 81.92 mV (`INA236_ADC_RANGE_81_92_MV`).

```

1  HAL_StatusTypeDef INA236_Init(INA236_HandleTypeDef *ina236, I2C_HandleTypeDef *i2c, uint8_t
    devAddress)
2  {
3      // Bind physical communication parameters to local handler
4      ina236->hi2c = i2c;
5      ina236->_devAddress = devAddress;
6
7      // 1. Verify if the sensor is physically responding on the I2C bus
8      if (HAL_I2C_IsDeviceReady(ina236->hi2c, (ina236->_devAddress) << 1, INA236_TRIALS,
        HAL_MAX_DELAY) != HAL_OK)
9      {
10         return HAL_ERROR;
11     }
12
13     // 2. Configure default settings
14     INA236_SetMode(ina236, INA236_CONTINUOUS_SHUNT_BUS_VOLTAGE);
15     INA236_SetShuntConvTime(ina236, INA236_1100_US);
16     INA236_SetBusConvTime(ina236, INA236_1100_US);

```

```

17 INA236_SetAverage(ina236, INA236_64_SAMPLES);
18 INA236_SetAdcRange(ina236, INA236_ADC_RANGE_81_92_MV);
19
20 return HAL_OK;
21 }

```

5.2.2 Función *INA236_RoundCurrentLsb* (Auxiliar Privada)

Aplica reglas matemáticas logarítmicas para redondear el paso mínimo del LSB de corriente obtenido a un valor decimal limpio de la serie de paso 1, 2 o 5.

- **Parámetros de Entrada:**

- lsbMin:** El LSB mínimo en Amperes por paso obtenido de la división de la corriente máxima de diseño entre la resolución del convertidor analógico a digital.

- **Retorno:** Valor de tipo double con el LSB redondeado definitivo.

- **Funcionamiento Interno:** La función obtiene la potencia de 10 inferior más cercana mediante el logaritmo base 10 del LSB mínimo:

$$\logLsb = \log_{10}(lsbMin) \quad (14)$$

$$powerOf10 = 10^{\lfloor \logLsb \rfloor} \quad (15)$$

Luego, normaliza el valor a un número real contenido entre 1.0 y 10.0 mediante:

$$normalized = \frac{lsbMin}{powerOf10} \quad (16)$$

Finalmente, evalúa el valor normalizado en un bloque de toma de decisiones condicionales para aproximarlos al paso inmediato superior en la serie 1, 2, 5 o 10, multiplicándolo nuevamente por la potencia de 10 de escala para restaurar el orden de magnitud decimal.

```

1 static double INA236_RoundCurrentLsb(double lsbMin)
2 {
3     // Find power of 10 below lsbMin
4     double logLsb = log10(lsbMin);
5     double powerOf10 = pow(10, floor(logLsb));
6
7     // Normalize to a value between 1.0 and 10.0
8     double normalized = lsbMin / powerOf10;
9     double roundedLsb;
10
11     // Round up to the nearest 1, 2, or 5 step
12     if (normalized <= 1.0)
13     {
14         roundedLsb = 1.0 * powerOf10;
15     }
16     else if (normalized <= 2.0)
17     {
18         roundedLsb = 2.0 * powerOf10;
19     }
20     else if (normalized <= 5.0)
21     {
22         roundedLsb = 5.0 * powerOf10;
23     }
24     else
25     {
26         roundedLsb = 10.0 * powerOf10;
27     }

```

```

28
29     return roundedLsb;
30 }

```

5.2.3 Función `INA236_SetCalibration`

Configura el valor resistivo del shunt y calcula e inyecta el coeficiente numérico de calibración de escala analógica en el registro respectivo del silicio del sensor.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura del driver.

rshuntvalue: Resistencia física del shunt soldado en la placa de hardware (expresada en Ohms).

maxcurrent: Máxima corriente de carga que se espera circule por la línea (expresada en Amperes).

- **Retorno:** Ninguno (void).

- **Funcionamiento Interno:** La función almacena localmente los parámetros de entrada y calcula el LSB de corriente absoluto de hardware dividiendo la corriente máxima esperada entre el rango analógico disponible en 15 bits (32768). Posteriormente, invoca a la función auxiliar privada `INA236_RoundCurrentLsb` para obtener un paso numérico amigable y limpio, salvando dicho valor en el miembro `_currentLsb`. A continuación, se computa el valor entero del registro de calibración aplicando la ecuación teórica del fabricante:

$$\text{SHUNT_CAL} = \frac{0.00512}{\text{roundedLsb} \times \text{rShuntValue}} \quad (17)$$

La función evalúa el estado del bit del rango dinámico del ADC. Si `_adcRange == 1` (que corresponde al rango analógico estrecho de entrada de ± 20.48 mV), divide el valor obtenido de la ecuación por 4 de forma obligatoria para ajustar la escala del multiplicador físico interno del chip. Finalmente, graba el coeficiente final en el registro `INA236_CALIBRATION_REGISTER` (0x05) llamando a `INA236_WriteRegister`.

```

1 void INA236_SetCalibration(INA236_HandleTypeDef *ina236, double rShuntValue, int maxCurrent)
2 {
3     ina236->_shuntResistor = rShuntValue;
4     ina236->_maximumCurrent = (double)maxCurrent;
5
6     // Calculate the minimum Current LSB: Max_Current / 2^15 (2^15 = 32768)
7     double currentLsbMinimum = (double)maxCurrent / 32768.0;
8     ina236->_currentLsbMin = currentLsbMinimum;
9
10    // Auto-calculate a clean, round LSB (always satisfies: lsbMin <= lsb < 2.5 * lsbMin)
11    double roundedLsb = INA236_RoundCurrentLsb(currentLsbMinimum);
12    ina236->_currentLsb = roundedLsb;
13
14    // Calculate the Calibration register value: 0.00512 / (roundedLsb * rShuntValue)
15    uint16_t shuntCal = 0.00512 / (roundedLsb * rShuntValue);
16
17    // According to datasheet, SHUNT_CAL must be divided by 4 for ADCRANGE = 1 (20.48 mV)
18    if (ina236->_adcRange == 1)
19    {
20        shuntCal = shuntCal / 4;
21    }
22
23    INA236_WriteRegister(ina236, INA236_CALIBRATION_REGISTER, shuntCal);
24 }

```

5.2.4 Función `INA236_ResetDevice`

Fuerza un reinicio de hardware en caliente por software sobre todos los registros internos de control del INA236.

- **Parámetros de Entrada:**
ina236: Puntero a la estructura del driver.
- **Retorno:** Ninguno.
- **Funcionamiento Interno:** La función escribe de forma directa la máscara de control `INA236_RST_Mask` (`0x8000`, correspondiente al bit 15 de control del registro de configuración `0x00`). Al activarse este bit, el procesador interno del chip borra toda su memoria volátil restaurando los parámetros predeterminados de fábrica. El bit se limpia de manera automática en el silicio. La función introduce una espera activa por hardware de 2 ms mediante `HAL_Delay` para garantizar la correcta estabilización del bus I2C y actualiza la bandera interna local del rango del ADC a su estado básico de ± 81.92 mV.

```

1 void INA236_ResetDevice(INA236_HandleTypeDef *ina236)
2 {
3     // Write directly the reset bit (bit 15 = 0x8000) to CONFIGURATION
4     INA236_WriteRegister(ina236, INA236_CONFIGURATION_REGISTER, INA236_RST_Mask);
5
6     // Wait a brief delay for internal circuits to settle
7     HAL_Delay(2);
8
9     // After reset, device returns to standard range +/- 81.92 mV
10    ina236->_adcRange = INA236_ADC_RANGE_81_92_MV;
11 }

```

5.3 Funciones de Configuración de Periféricos

Estas funciones permiten modificar de manera selectiva y segura campos específicos de bits contenidos en el registro `CONFIGURATION` (`0x00`), evitando alterar los valores configurados en otros campos adyacentes del mismo registro.

5.3.1 Función `INA236_SetMode`

Modifica el modo de operación del convertidor analógico a digital del sensor.

- **Parámetros de Entrada:**
ina236: Puntero a la estructura del driver.
mode: Modo seleccionado de tipo enumerado `INA236_Mode_TypeDef` (por ejemplo, continuo, apagado o disparo único).
- **Funcionamiento Interno:** Aplica la técnica de lectura-modificación-escritura:
 1. Lee el registro de configuración de 16 bits llamando a `INA236_ReadRegister`.
 2. Limpia los bits del modo (bits 0, 1 y 2) aplicando una operación lógica AND con la máscara del modo invertida:

$$\text{regValue} = \text{regValue} \& (\sim \text{INA236_MODE_Mask}) \quad (18)$$

3. Carga el nuevo modo de operación alineando los bits mediante un desplazamiento a su posición y protegiéndolos mediante un AND lógico con la máscara antes de sumarlos con un OR lógico:

$$\text{regValue} = \text{regValue} | ((\text{mode} \ll \text{INA236_MODE_Pos}) \& \text{INA236_MODE_Mask}) \quad (19)$$

4. Escribe el resultado final de vuelta en el sensor mediante INA236_WriteRegister.

```

1 void INA236_SetMode(INA236_HandleTypeDef *ina236, INA236_Mode_TypeDef mode)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CONFIGURATION_REGISTER);
4     regValue &= ~INA236_MODE_Mask;
5     regValue |= (mode << INA236_MODE_Pos) & INA236_MODE_Mask;
6     INA236_WriteRegister(ina236, INA236_CONFIGURATION_REGISTER, regValue);
7 }

```

5.3.2 Función INA236_SetShuntConvTime

Configura el tiempo de conversión e integración de tensión diferencial en la entrada del shunt analógico.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura del driver.

convtime: Tiempo de conversión seleccionado de la enumeración de hardware INA236_ConvTime_TypeDef (rango de 140 µs a 8.24 ms).

- **Funcionamiento Interno:** Modifica de manera selectiva los bits 3, 4 y 5 del registro de configuración. Utiliza la máscara específica INA236_VSHCT_Mask (0x0038) y realiza el desplazamiento lógico mediante INA236_VSHCT_Pos (3U), siguiendo el método de lectura-modificación-escritura.

```

1 void INA236_SetShuntConvTime(INA236_HandleTypeDef *ina236, INA236_ConvTime_TypeDef convTime)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CONFIGURATION_REGISTER);
4     regValue &= ~INA236_VSHCT_Mask;
5     regValue |= (convTime << INA236_VSHCT_Pos) & INA236_VSHCT_Mask;
6     INA236_WriteRegister(ina236, INA236_CONFIGURATION_REGISTER, regValue);
7 }

```

5.3.3 Función INA236_SetBusConvTime

Configura el tiempo de conversión del ADC para la línea común del bus de tensión (VBUS).

- **Parámetros de Entrada:**

ina236: Puntero a la estructura del driver.

convtime: Constante de tiempo seleccionada del tipo enumerado INA236_ConvTime_TypeDef.

- **Funcionamiento Interno:** Aísla y modifica los bits 6, 7 y 8 de la palabra de configuración física utilizando la máscara INA236_VBUSCT_Mask (0x01C0) y la constante de posición de desplazamiento INA236_VBUSCT_Pos (6U).

```

1 void INA236_SetBusConvTime(INA236_HandleTypeDef *ina236, INA236_ConvTime_TypeDef convTime)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CONFIGURATION_REGISTER);
4     regValue &= ~INA236_VBUSCT_Mask;
5     regValue |= (convTime << INA236_VBUSCT_Pos) & INA236_VBUSCT_Mask;
6     INA236_WriteRegister(ina236, INA236_CONFIGURATION_REGISTER, regValue);
7 }

```


5.3.4 Función `INA236_SetAverage`

Establece el número de muestras consecutivas que el conversor del chip promediará internamente antes de actualizar sus registros de datos y notificar la bandera de datos listos.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura del driver.

avg: Cantidad de promedios de la enumeración `INA236_Avg_TypeDef` (valores admitidos de 1 a 1024 muestras).

- **Funcionamiento Interno:** Modifica los bits 9, 10 y 11 del registro de configuración utilizando la máscara `INA236_AVG_Mask` (0x0E00) y la constante de desplazamiento `INA236_AVG_Pos` (9U).

```

1 void INA236_SetAverage(INA236_HandleTypeDef *ina236, INA236_Avg_TypeDef avg)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CONFIGURATION_REGISTER);
4     regValue &= ~INA236_AVG_Mask;
5     regValue |= (avg << INA236_AVG_Pos) & INA236_AVG_Mask;
6     INA236_WriteRegister(ina236, INA236_CONFIGURATION_REGISTER, regValue);
7 }

```

5.3.5 Función `INA236_SetAdcRange`

Configura el rango dinámico de tensión en la entrada analógica del convertidor diferencial de shunt y actualiza las constantes de resolución matemática del firmware.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura del driver.

range: Rango de entrada física del ADC de la enumeración `INA236_AdcRange_TypeDef` (0 para ± 81.92 mV o 1 para ± 20.48 mV).

- **Funcionamiento Interno:** El bit 12 de configuración es aislado y modificado mediante la máscara `INA236_ADCRANGE_Mask` (0x1000) y la posición `INA236_ADCRANGE_Pos` (12U). Adicionalmente, actualiza el valor lógico `_adcRange` de la estructura de datos local y recalcula la resolución de paso analógico a digital (`_resolution`) y el límite del rango dinámico (`_shuntAdcRange`) que serán usados por las funciones de cálculo de punto flotante en el microcontrolador:

- Si se selecciona el rango estrecho de ± 20.48 mV, la resolución física del ADC se fija en 625 nV/LSB (0.000000625).
- Si se selecciona el rango ancho de ± 81.92 mV, la resolución física se establece en 2.5 μ V/LSB (0.0000025).

```

1 void INA236_SetAdcRange(INA236_HandleTypeDef *ina236, INA236_AdcRange_TypeDef range)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CONFIGURATION_REGISTER);
4     regValue &= ~INA236_ADCRANGE_Mask;
5     regValue |= (range << INA236_ADCRANGE_Pos) & INA236_ADCRANGE_Mask;
6     INA236_WriteRegister(ina236, INA236_CONFIGURATION_REGISTER, regValue);
7
8     ina236->_adcRange = (_Bool)range;
9     if (range == INA236_ADC_RANGE20_48_MV)
10     {
11         ina236->_shuntAdcRange = 0.02048;
12         ina236->_resolution = 0.000000625; // 625 nV/LSB
13     }

```



```

14     else
15     {
16         ina236->_shuntAdcRange = 0.08192;
17         ina236->_resolution = 0.0000025;    // 2.5 uV/LSB
18     }
19 }

```

5.4 Funciones de Gestión de Alertas por Hardware

Estas funciones interactúan de forma directa sobre el registro de máscara de interrupciones y habilitación de alertas MASK_ENABLE (0x06) para programar la lógica del pin físico ALERT.

5.4.1 Configuración de Polaridad del Pin ALERT

Define la lógica eléctrica del pin físico ALERT del sensor en modo normalmente abierto activo bajo u opcionalmente activo alto.

- **Funcionamiento Interno:** La función de escritura manipula el bit 1 (APOL) del registro de máscara mediante INA236_APOL_Mask. La función de lectura lee el registro esclavo, verifica si no ocurrió un error I2C (en cuyo caso retorna la polaridad segura por defecto de activo bajo) y extrae el estado físico realizando una operación lógica AND y un desplazamiento a la derecha para obtener un valor limpio de 0 o 1 que coincide con las definiciones de la enumeración:

$$\text{Retorno} = \frac{\text{regValue} \& \text{INA236_APOL}}{\text{INA236_APOL_Pos}} \quad (20)$$

La función además imprime un diagnóstico legible en español mediante UART a través de la consola de depuración del sistema utilizando la función estándar printf.

```

1 void INA236_SetAlertPolarity(INA236_HandleTypeDef *ina236, INA236_AlertPol_TypeDef polarity)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
4     regValue &= ~INA236_APOL_Mask;
5     regValue |= (polarity << INA236_APOL_Pos) & INA236_APOL_Mask;
6     INA236_WriteRegister(ina236, INA236_MASK_ENABLE_REGISTER, regValue);
7 }
8
9 INA236_AlertPol_TypeDef INA236_GetAlertPolarity(INA236_HandleTypeDef *ina236)
10 {
11     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
12     if (regValue == 0xFFFF)
13     {
14         return INA236_ACTIVE_LOW;
15     }
16     INA236_AlertPol_TypeDef polarity = (INA236_AlertPol_TypeDef)((regValue & INA236_APOL) >>
17         INA236_APOL_Pos);
18     if (polarity == INA236_ACTIVE_HIGH)
19     {
20         printf("Alert pin is configured as ACTIVE-HIGH\r\n");
21     }
22     else
23     {
24         printf("Alert pin is configured as ACTIVE-LOW\r\n");
25     }
26     return polarity;
27 }

```

5.4.2 Configuración de Alerta por Conversión Completa

Asigna la conmutación física del pin ALERT para notificar externamente al microcontrolador cada vez que el ADC del sensor termina una conversión analógica a digital.

- **Funcionamiento Interno:** Modifica el bit 10 (CNVR) del registro de máscara mediante INA236_CNVR_Mask y la posición INA236_CNVR_Pos (10U). La función de consulta lee el registro, extrae el bit mediante desplazamiento lógico a la derecha por 10 posiciones e imprime un aviso en consola informando al usuario sobre la configuración actual del pin de alerta.

```

1 void INA236_SetAlertPin(INA236_HandleTypeDef *ina236, INA236_ConvRdy_TypeDef cnvr)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
4     regValue &= ~INA236_CNVR_Mask;
5     regValue |= (cnvr << INA236_CNVR_Pos) & INA236_CNVR_Mask;
6     INA236_WriteRegister(ina236, INA236_MASK_ENABLE_REGISTER, regValue);
7 }
8
9 INA236_ConvRdy_TypeDef INA236_GetAlertPin(INA236_HandleTypeDef *ina236)
10 {
11     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
12     if (regValue == 0xFFFF)
13     {
14         printf("ERROR - ALERT pin does not assert on conversion ready\r\n");
15         return INA236_CONV_READY_ALERT_DISABLE;
16     }
17     INA236_ConvRdy_TypeDef alert = (INA236_ConvRdy_TypeDef)((regValue & INA236_CNVR) >>
18     INA236_CNVR_Pos);
19     if (alert == INA236_CONV_READY_ALERT_ENABLE)
20     {
21         printf("ALERT pin asserts when conversion is ready\r\n");
22     }
23     else
24     {
25         printf("ALERT pin does not assert on conversion ready\r\n");
26     }
27     return alert;
28 }

```

5.4.3 Funciones de Selección de Límites de Alerta

Estas funciones configuran los bits superiores de habilitación en el registro de máscara para que el chip compare internamente y dispare el pin físico de alerta si las lecturas de entrada superan los límites configurados por el firmware en el registro de límite de alerta ALERT_LIMIT (0x07). Para evitar conflictos de configuración lógica en el registro de alertas/máscara (donde solo debe haber una alerta de límite activa a la vez), cada una de estas funciones implementa una lógica de exclusión mutua cuando es habilitada:

1. Almacena el tipo de alerta activa en el miembro _alertType de la estructura de control de tipo INA236_AlertType_TypeDef. Esto permite a la función INA236_SetAlertLimit conocer el tipo de escalado físico a aplicar sobre el umbral del registro de límite.
2. Limpia a nivel de bit las otras cuatro alertas de límites en el registro MASK_ENABLE (poniéndolas en 0).
3. Habilita su bit correspondiente en el registro MASK_ENABLE (poniéndolo en 1).

Si la alerta se configura como deshabilitada, la función simplemente limpia su propio bit correspondiente en el registro sin alterar los demás bits.

A continuación se detalla la implementación física de estas cinco funciones en el archivo INA236.c:

```

1 void INA236_SetPowerOverLimit(INA236_HandleTypeDef *ina236, INA236_PowerAlert_TypeDef
2   powerAlert)
3 {
4     // Read the current MASK_ENABLE register (Address: 0x06)
5     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
6
7     if (powerAlert == INA236_POWER_LIMIT_ALERT_ENABLE)
8     {
9         // Set the active alert type in the handler structure
10        ina236->_alertType = INA236_POL_ALERT;
11
12        // Clear bits 15 (SOL), 14 (SUL), 13 (BOL), and 12 (BUL) by setting them to 0
13        regValue &= ~(INA236_SOL | INA236_SUL | INA236_BOL | INA236_BUL);
14
15        // Set bit 11 (POL) to 1
16        regValue |= INA236_POL;
17    }
18    else
19    {
20        // If polAlert is disabled, just clear bit 11 (POL)
21        regValue &= ~INA236_POL;
22    }
23
24    // Write the updated value back to the MASK_ENABLE register
25    INA236_WriteRegister(ina236, INA236_MASK_ENABLE_REGISTER, regValue);
26 }
27
28 void INA236_SetBusUnderLimit(INA236_HandleTypeDef *ina236, INA236_BusUnderLimitAlert_TypeDef
29   bulAlert)
30 {
31     // Read the current MASK_ENABLE register (Address: 0x06)
32     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
33
34     if (bulAlert == INA236_BUS_UNDER_LIMIT_ALERT_ENABLE)
35     {
36         // Set the active alert type in the handler structure
37         ina236->_alertType = INA236_BUL_ALERT;
38
39        // Clear bits 15 (SOL), 14 (SUL), 13 (BOL), and 11 (POL) by setting them to 0
40        regValue &= ~(INA236_SOL | INA236_SUL | INA236_BOL | INA236_POL);
41
42        // Set bit 12 (BUL) to 1
43        regValue |= INA236_BUL;
44    }
45    else
46    {
47        // If bulAlert is disabled, just clear bit 12 (BUL)
48        regValue &= ~INA236_BUL;
49    }
50
51    // Write the updated value back to the MASK_ENABLE register
52    INA236_WriteRegister(ina236, INA236_MASK_ENABLE_REGISTER, regValue);
53 }
54
55 void INA236_SetBusOverLimit(INA236_HandleTypeDef *ina236, INA236_BusOverLimitAlert_TypeDef
56   bolAlert)
57 {
58     // Read the current MASK_ENABLE register (Address: 0x06)
59     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);

```

```

58     if (bolAlert == INA236_BUS_OVER_LIMIT_ALERT_ENABLE)
59     {
60         // Set the active alert type in the handler structure
61         ina236->_alertType = INA236_BOL_ALERT;
62
63         // Clear bits 15 (SOL), 14 (SUL), 12 (BUL), and 11 (POL) by setting them to 0
64         regValue &= ~(INA236_SOL | INA236_SUL | INA236_BUL | INA236_POL);
65
66         // Set bit 13 (BOL) to 1
67         regValue |= INA236_BOL;
68     }
69     else
70     {
71         // If bolAlert is disabled, just clear bit 13 (BOL)
72         regValue &= ~INA236_BOL;
73     }
74
75     // Write the updated value back to the MASK_ENABLE register
76     INA236_WriteRegister(ina236, INA236_MASK_ENABLE_REGISTER, regValue);
77 }
78
79 void INA236_SetShuntUnderLimit(INA236_HandleTypeDef *ina236,
80     INA236_ShuntUnderLimitAlert_TypeDef sulAlert)
81 {
82     // Read the current MASK_ENABLE register (Address: 0x06)
83     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
84
85     if (sulAlert == INA236_SHUNT_UNDER_LIMIT_ALERT_ENABLE)
86     {
87         // Set the active alert type in the handler structure
88         ina236->_alertType = INA236_SUL_ALERT;
89
90         // Clear bits 15 (SOL), 13 (BOL), 12 (BUL), and 11 (POL) by setting them to 0
91         regValue &= ~(INA236_SOL | INA236_BOL | INA236_BUL | INA236_POL);
92
93         // Set bit 14 (SUL) to 1
94         regValue |= INA236_SUL;
95     }
96     else
97     {
98         // If sulAlert is disabled, just clear bit 14 (SUL)
99         regValue &= ~INA236_SUL;
100     }
101
102     // Write the updated value back to the MASK_ENABLE register
103     INA236_WriteRegister(ina236, INA236_MASK_ENABLE_REGISTER, regValue);
104 }
105
106 void INA236_SetShuntOverLimit(INA236_HandleTypeDef *ina236,
107     INA236_ShuntOverLimitAlert_TypeDef solAlert)
108 {
109     // Read the current MASK_ENABLE register (Address: 0x06)
110     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
111
112     if (solAlert == INA236_SHUNT_OVER_LIMIT_ALERT_ENABLE)
113     {
114         // Set the active alert type in the handler structure
115         ina236->_alertType = INA236_SOL_ALERT;
116
117         // Clear bits 14 (SUL), 13 (BOL), 12 (BUL), and 11 (POL) by setting them to 0

```

```

116     regValue &= ~(INA236_SUL | INA236_BOL | INA236_BUL | INA236_POL);
117
118     // Set bit 15 (SOL) to 1
119     regValue |= INA236_SOL;
120 }
121 else
122 {
123     // If solAlert is disabled, just clear bit 15 (SOL)
124     regValue &= ~INA236_SOL;
125 }
126
127 // Write the updated value back to the MASK_ENABLE register
128 INA236_WriteRegister(ina236, INA236_MASK_ENABLE_REGISTER, regValue);
129 }

```

5.4.4 Función `INA236_SetAlertLimit`

Esta función configura el valor de comparación física (umbral) en el registro `ALERT_LIMIT` (0x07) del sensor.

- **Parámetros de Entrada:**

ina236: Puntero a la estructura de control `INA236_HandleTypeDef`. Utiliza el miembro `_alertType` para conocer el tipo de alerta activa y el miembro `_resolution` o `_currentLsb` para aplicar el factor de conversión correspondiente.

alertLimit: Valor físico de tipo `double` que representa el límite deseado en unidades reales:

- En **Voltios (V)** para alertas de tensión del shunt (`SOL_ALERT` o `SUL_ALERT`).
- En **Voltios (V)** para alertas de tensión del bus (`BOL_ALERT` o `BUL_ALERT`).
- En **Vatios (W)** para alertas de límite de potencia (`POL_ALERT`).

- **Retorno:** Ninguno (`void`).

- **Funcionamiento Interno:** La función implementa un escalado matemático automático dependiente de la variable de estado `_alertType` configurada por las funciones de selección de límites de alerta:

1. **Alertas de tensión del shunt (`SOL_ALERT` o `SUL_ALERT`):** La tensión física de alerta en Voltios se divide por la resolución del shunt del ADC (en Voltios/LSB) para obtener el valor binario entero correspondiente:

$$\text{rawValue} = \frac{\text{alertLimit}}{\text{ina236} \rightarrow _resolution} \quad (21)$$

2. **Alertas de tensión de bus (`BOL_ALERT` o `BUL_ALERT`):** El registro de tensión de bus del INA236 posee una resolución de paso fija de 1.6 mV/LSB (0.0016 V/LSB). Por lo tanto, el umbral de tensión de bus en Voltios se divide por esta constante:

$$\text{rawValue} = \frac{\text{alertLimit}}{0.0016} \quad (22)$$

3. **Alerta de límite de potencia (`POL_ALERT`):** Dado que la resolución de la potencia está acoplada al LSB de corriente de diseño por una constante de escala de 32, el límite de potencia física en Vatios se divide por dicha constante multiplicada por el paso de corriente final:

$$\text{rawValue} = \frac{\text{alertLimit}}{32.0 \times \text{ina236} \rightarrow _currentLsb} \quad (23)$$

Posteriormente, el valor crudo calculado (`rawValue`) se escribe en el registro físico `INA236_ALERT_LIMIT_REGISTER` (0x07) mediante la función de bajo nivel `INA236_WriteRegister`.

```

1 void INA236_SetAlertLimit(INA236_HandleTypeDef *ina236, double alertLimit)
2 {
3     uint16_t rawValue = 0;
4
5     // Shunt voltage alerts (SOL, SUL) - scaled by shunt ADC resolution (in Volts)
6     if (ina236->_alertType == INA236_SOL_ALERT || ina236->_alertType == INA236_SUL_ALERT)
7     {
8         rawValue = (uint16_t)(alertLimit / ina236->_resolution);
9     }
10    // Bus voltage alerts (BOL, BUL) - fixed 1.6 mV/LSB resolution
11    else if (ina236->_alertType == INA236_BOL_ALERT || ina236->_alertType ==
12             INA236_BUL_ALERT)
13    {
14        rawValue = (uint16_t)(alertLimit / 0.0016);
15    }
16    // Power over limit alert (POL) - fixed 32 * Current_LSB resolution
17    else if (ina236->_alertType == INA236_POL_ALERT)
18    {
19        rawValue = (uint16_t)(alertLimit / (32.0 * ina236->_currentLsb));
20    }
21
22    // Write the raw value to the Alert Limit Register (Address: 0x07)
23    INA236_WriteRegister(ina236, INA236_ALERT_LIMIT_REGISTER, rawValue);
24 }

```

5.5 Funciones de Lectura de Diagnóstico e Identificación

Estas funciones monitorean el estado de las banderas internas de conversión de datos por software (polleo de estados) y permiten comprobar la validez del hardware leyendo los códigos únicos grabados en fábrica.

5.5.1 Lectura de Banderas de Conversión y Desbordamiento

Permiten consultar si existen datos listos de tensión/corriente para lectura o si ha ocurrido un error matemático crítico.

- **Funcionamiento Interno:**

ina236_isconversionready: Consulta el estado de actualización del ADC aislando el bit 3 (CVRF) del registro de máscara. Devuelve 1 si hay una nueva conversión lista para ser leída. La bandera se borra automáticamente al leer el registro o al escribir en configuración.

ina236_ismathoverflowready: Consulta si ocurrió una saturación analógica o desbordamiento en el coprocesador matemático del chip leyendo el bit 2 (OVF). Si el bit es 1, las lecturas matemáticas internas de corriente y potencia no son válidas por desbordamiento y deben descartarse.

```

1 _Bool INA236_IsConversionReady(INA236_HandleTypeDef *ina236)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
4     if (regValue == 0xFFFF)
5     {
6         return 0;
7     }
8     if ((regValue & INA236_CVRF) != 0U)
9     {
10        return 1; // Conversion Ready
11    }
12 }

```

```

11     }
12     return 0;
13 }
14
15 uint8_t INA236_IsMathOverflowReady(INA236_HandleTypeDef *ina236)
16 {
17     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
18     if (regValue == 0xFFFF)
19     {
20         return 0;
21     }
22     if ((regValue & INA236_OVF) != 0U)
23     {
24         return 1; // Math Overflow occurred
25     }
26     return 0;
27 }

```

5.5.2 Lectura de Identificadores del Dispositivo

Verifica la firma única de fabricante y dispositivo de silicio para control de calidad.

- **Funcionamiento Interno:** Estas funciones leen directamente los registros finales del mapa de direcciones del INA236: el ID del fabricante en la dirección 0x3E (que debe retornar siempre la constante de Texas Instruments 0x5449) y el ID de dispositivo en la dirección 0x3F (que debe retornar la firma del modelo INA236 0xA080).

```

1 uint16_t INA236_GetManufacturerID(INA236_HandleTypeDef *ina236)
2 {
3     return INA236_ReadRegister(ina236, MANUFACTURER_ID_REGISTER);
4 }
5
6 uint16_t INA236_GetDeviceID(INA236_HandleTypeDef *ina236)
7 {
8     return INA236_ReadRegister(ina236, DEVICE_ID_REGISTER);
9 }

```

5.5.3 Lectura de Registros Internos de Configuración y Calibración

Estas funciones permiten leer de forma directa los valores enteros crudos de 16 bits almacenados en los registros internos clave del integrado para fines de depuración o validación de estado por software:

- **Funcionamiento Interno:**
 - ina236_getconfiguration:** Lee el registro de configuración física INA236_CONFIGURATION_REGISTER (0x00) y retorna su valor de control de 16 bits.
 - ina236_getcalibration:** Lee el registro de calibración física INA236_CALIBRATION_REGISTER (0x05) que contiene el factor multiplicador de corriente.
 - ina236_getmaskenable:** Lee el registro de alertas y habilitación INA236_MASK_ENABLE_REGISTER (0x06).

Todas estas funciones retornan el valor directo de 16 bits sin signo (uint16_t) y devuelven el valor de error 0xFFFF si la comunicación por el bus I2C falla física o lógicamente.

```

1 uint16_t INA236_GetConfiguration(INA236_HandleTypeDef *ina236)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CONFIGURATION_REGISTER);

```

```

4     return regValue;
5 }
6
7
8 uint16_t INA236_GetCalibration(INA236_HandleTypeDef *ina236)
9 {
10     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CALIBRATION_REGISTER);
11
12     return regValue;
13 }
14
15 uint16_t INA236_GetMaskEnable(INA236_HandleTypeDef *ina236)
16 {
17     uint16_t regValue = INA236_ReadRegister(ina236, INA236_MASK_ENABLE_REGISTER);
18
19     return regValue;
20 }

```

5.6 Funciones de Instrumentación y Medida Física

Estas funciones implementan las conversiones matemáticas definitivas. Extraen los datos crudos firmados en enteros de dos complementos desde los registros de medida y los escalan utilizando multiplicadores en punto flotante (double) para entregárselos a la aplicación.

5.6.1 Lectura de Tensión de Shunt

Devuelve el voltaje diferencial en bornes del shunt medido por el ADC, expresado en millivolts (mV).

- **Funcionamiento Interno:** Lee el registro SHUNT_VOLTAGE_REGISTER (0x01). Convierte el entero sin signo recibido a un tipo de entero de 16 bits firmado de manera explícita:

$$\text{rawValue} = (\text{int16_t})\text{regValue} \quad (24)$$

Esto preserva de forma nativa la firma bidireccional de dos complementos de la corriente en la resistencia (los valores negativos indican flujo inverso de carga). Finalmente, calcula los millivolts reales multiplicando por la resolución física configurada en el ADC (_resolution en Volts) y por un factor de escala de 1000.0:

$$\text{Retorno} = \text{rawValue} \times \text{ina236} \rightarrow _resolution \times 1000.0 \quad (25)$$

```

1 double INA236_GetShuntVoltage_mV(INA236_HandleTypeDef *ina236)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_SHUNT_VOLTAGE_REGISTER);
4     int16_t rawValue = (int16_t)regValue;
5     return (double)rawValue * ina236->_resolution * 1000.0;
6 }

```

5.6.2 Lectura de Tensión de Bus

Devuelve el voltaje real del bus de alimentación medido en la terminal de carga, expresado en Volts (V).

- **Funcionamiento Interno:** Lee el registro BUS_VOLTAGE_REGISTER (0x02). Al tratarse de una tensión de bus de alimentación que no puede ser negativa en polaridad normal, el valor se procesa como entero sin signo. Para evitar que un error de comunicación I2C (que devuelve 0xFFFF) sea interpretado como una lectura válida gigante de 104.8 V, se valida si

regValue == 0xFFFF para retornar en ese caso 0.0. De lo contrario, escala el valor de los LSB leídos por el paso constante interno de hardware del integrado de 1.6 mV (0.0016 V):

$$V_{\text{bus}} [\text{V}] = \text{regValue} \times 0.0016 \quad (26)$$

```

1 double INA236_GetBusVoltage(INA236_HandleTypeDef *ina236)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_BUS_VOLTAGE_REGISTER);
4     if (regValue == 0xFFFF)
5     {
6         return 0.0;
7     }
8     return (double)regValue * 0.0016;
9 }

```

5.6.3 Lectura de Corriente

Devuelve la corriente eléctrica instantánea calculada que circula a través de la carga, expresada en Amperes (A).

- **Funcionamiento Interno:** Lee el registro interno CURRENT_REGISTER (0x04). Si ocurre un fallo en el bus de datos (retorno 0xFFFF), devuelve de forma segura 0.0. En caso de éxito, convierte la lectura a un entero firmado de 16 bits de dos complementos para conservar el signo del flujo, y multiplica el valor crudo por el LSB de corriente seleccionado y redondeado en la fase de calibración:

$$I [\text{A}] = (\text{int16_t})\text{regValue} \times \text{ina236} \rightarrow _currentLsb \quad (27)$$

```

1 double INA236_GetCurrent_A(INA236_HandleTypeDef *ina236)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_CURRENT_REGISTER);
4     if (regValue == 0xFFFF) return 0.0;
5     return (double)((int16_t)regValue) * ina236->_currentLsb;
6 }

```

5.6.4 Lectura de Potencia Consumida

Devuelve la potencia eléctrica disipada en la carga por el sistema, expresada en Watts (W).

- **Funcionamiento Interno:** Lee el registro POWER_REGISTER (0x03). La potencia siempre se almacena en enteros positivos unipolar sin signo. La función comprueba errores I2C y, en condiciones normales, escala el registro leído aplicando el multiplicador físico definido por la arquitectura de silicio del INA236: el paso de la potencia es equivalente a 32 veces el LSB de la corriente:

$$P [\text{W}] = \text{regValue} \times (32.0 \times \text{ina236} \rightarrow _currentLsb) \quad (28)$$

```

1 double INA236_GetPower_W(INA236_HandleTypeDef *ina236)
2 {
3     uint16_t regValue = INA236_ReadRegister(ina236, INA236_POWER_REGISTER);
4     if (regValue == 0xFFFF) return 0.0;
5     // Power LSB is fixed at 32 * Current_LSB
6     return (double)regValue * (32.0 * ina236->_currentLsb);
7 }

```

6 INTEGRACIÓN DE LA LIBRERÍA EN CMAKE Y SOPORTE DE PUNTO FLOTANTE

Para utilizar de forma permanente la librería del sensor INA236 en un desarrollo basado en CMake en Visual Studio Code, se deben integrar sus fuentes de forma explícita y configurar el soporte físico para punto flotante en el microcontrolador.

6.1 Modificación del archivo CMakeLists.txt

Las cabeceras y fuentes agregadas a mano deben vincularse en las secciones correspondientes de la siguiente manera:

```

1 # Add sources to executable
2 target_sources(${CMAKE_PROJECT_NAME} PRIVATE
3     # Add user sources here
4     Core/INA236/Src/INA236.c
5 )
6
7 # Add include paths
8 target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
9     # Add user defined include paths
10    Core/INA236/Inc
11 )

```

6.2 Habilitación de Punto Flotante en la Toolchain (Newlib-nano)

Por defecto, la compilación de proyectos embebidos para STM32 utiliza la versión de librería estándar ligera denominada **Newlib-nano**. Para ahorrar memoria Flash y RAM en microcontroladores con recursos críticos, Newlib-nano deshabilita de fábrica el soporte para dar formato de salida a variables de tipo flotante (float o double) mediante llamadas a funciones clásicas como `sprintf` o `printf`.

Si intenta imprimir el valor decimal devuelto por `INA236_GetCurrent_A()` con la sentencia:

```

1 char msgBuffer[100];
2 sprintf(msgBuffer, "Corriente: %.3f A\r\n", currentVal);

```

El búfer resultante omitirá los decimales o mostrará un signo de interrogación vacío en la terminal serie.

Para forzar al compilador de GCC a enlazar las librerías matemáticas completas y permitir la conversión de datos decimales, se deben activar explícitamente las opciones de enlazador en el archivo `CMakeLists.txt`. Busque la sección del bloque `target_link_options` y agregue las banderas correspondientes como se muestra a continuación:

```

1 target_link_options(${CMAKE_PROJECT_NAME} PRIVATE
2     -u_printf_float # Habilita el soporte para formatear flotantes en sprintf y printf
3     -u_scanf_float  # Habilita el soporte para recibir flotantes en sscanf y scanf
4 )

```

Costo en Memoria de las Banderas de Flotantes

Activar estas dos directivas matemáticas agrega aproximadamente entre 8 KB y 12 KB adicionales al tamaño final del archivo binario compilado en la Flash del procesador. Es muy importante comprobar el uso de recursos desde la pestaña **STM32 Build Analyzer** de VS Code para asegurar que no exceda los límites de memoria física del procesador.

ANEXO: MANIPULACIÓN DE BITS EN LENGUAJE C PARA SISTEMAS EMBEBIDOS

En el desarrollo de firmware para sistemas embebidos, la manipulación de registros a nivel de bit es una operación fundamental y cotidiana. Dado que el hardware físico mapea periféricos y configuraciones a registros de memoria de tamaño fijo (habitualmente de 8, 16 o 32 bits), el programador debe ser capaz de modificar selectivamente bits individuales o grupos de bits sin alterar el estado del resto del registro.

A continuación, se presenta una guía detallada con ejemplos en lenguaje C sobre las operaciones de bits más comunes.

1. Establecer un bit a 1 (Bit Set)

Para forzar a nivel alto (1) un bit en la posición n de una variable o registro sin modificar los demás bits, se utiliza el operador lógico de suma **OR a nivel de bits** (`|`) en combinación con un desplazamiento de bits hacia la izquierda (`<<`) para crear la máscara:

$$\text{registro} = \text{registro} | (1 \ll n) \quad (29)$$

En C, la expresión compacta de asignación es:

```
1 // Establece a 1 el bit en la posición 'n'
2 registro |= (1U << n);
```

Ejemplo práctico: Si deseamos activar el bit de reset por software en el registro de configuración del INA236 (que se encuentra en la posición 15):

```
1 #define INA236_RST_Pos 15U
2 uint16_t regValue = 0x4127; // Estado inicial
3
4 // Establecer el bit 15 a 1
5 regValue |= (1U << INA236_RST_Pos);
6 // regValue ahora es 0xC127 (bit 15 en 1)
```

2. Limpiar un bit a 0 (Bit Clear)

Para forzar a nivel bajo (0) un bit en la posición n sin alterar el resto de la variable, se combina el operador lógico de producto **AND a nivel de bits** (`&`) con el operador de negación lógica **NOT a nivel de bits** (`~`) sobre la máscara de bits:

$$\text{registro} = \text{registro} \& \sim (1 \ll n) \quad (30)$$

En C, la sintaxis estándar es:

```
1 // Limpia a 0 el bit en la posición 'n'
2 registro &= ~(1U << n);
```

Ejemplo práctico: Para deshabilitar el bit de alerta de sobretensión en shunt (SOL, en la posición 15) en el registro de máscara del INA236:

```
1 #define INA236_SOL_Pos 15U
2 uint16_t regValue = 0x8002; // Alerta SOL habilitada
3
4 // Limpiar el bit 15 a 0
5 regValue &= ~(1U << INA236_SOL_Pos);
6 // regValue ahora es 0x0002 (bit 15 en 0)
```

3. Establecer un conjunto de bits a 1

Cuando es necesario forzar simultáneamente a 1 un grupo de bits no contiguos o contiguos, se genera una máscara combinada uniendo las posiciones de los bits mediante el operador OR a nivel de bits (`|`). Posteriormente, se aplica el operador OR sobre el registro:

```
1 // Establece a 1 un conjunto de bits definidos por sus mascarar
2 registro |= (MASK_A | MASK_B | MASK_C);
```

Ejemplo práctico: Si deseamos habilitar al mismo tiempo las alertas de sobrecorriente en shunt (SOL, bit 15) y de sobretensión en bus (BOL, bit 13):

```
1 #define INA236_SOL (1U << 15U)
2 #define INA236_BOL (1U << 13U)
3 uint16_t regValue = 0x0002; // Estado inicial
4
5 // Activar bits 15 y 13
6 regValue |= (INA236_SOL | INA236_BOL);
7 // regValue ahora es 0xA002 (bits 15 y 13 en 1)
```

4. Limpiar a 0 un conjunto de bits

Para forzar a 0 un conjunto de bits al mismo tiempo, primero se realiza la unión de las máscaras de dichos bits mediante una operación lógica OR (`|`). Después, se niega el resultado de esta máscara combinada mediante NOT (`~`) y se aplica una operación lógica AND (`&`) sobre el registro:

```
1 // Limpia a 0 un conjunto de bits a partir de sus mascarar
2 registro &= ~(MASK_A | MASK_B | MASK_C);
```

Ejemplo práctico: Este procedimiento se utiliza en el controlador del INA236 para garantizar la exclusión mutua de alertas. Por ejemplo, al habilitar la alerta SOL, debemos borrar los otros tipos de alerta (SUL en bit 14, BOL en bit 13, BUL en bit 12 y POL en bit 11) en el registro de control:

```
1 #define INA236_SUL (1U << 14U)
2 #define INA236_BOL (1U << 13U)
3 #define INA236_BUL (1U << 12U)
4 #define INA236_POL (1U << 11U)
5 uint16_t regValue = 0x7802; // Bits 14, 13, 12 y 11 activos
6
7 // Limpiar a 0 los bits 14, 13, 12 y 11
8 regValue &= ~(INA236_SUL | INA236_BOL | INA236_BUL | INA236_POL);
9 // regValue ahora es 0x0002 (todos los bits anteriores en 0)
```

5. Leer el estado de un bit

Para determinar si un bit en la posición `n` se encuentra en nivel alto (1) o nivel bajo (0), se realiza una operación lógica AND (`&`) entre el registro y la máscara del bit. Existen dos alternativas comunes para retornar su valor:

Alternativa A: Retornar el bit aislado en su posición original

```
1 // Retorna 0 si el bit es 0, o (1 << n) si el bit es 1
2 uint16_t bitStatus = (registro & (1U << n));
```

Alternativa B: Desplazar el bit a la posición del bit 0 (Lógica booleana de 0 o 1)

```
1 // Desplaza a la derecha y aísla para retornar exactamente 0 o 1
2 uint8_t bitValue = (uint8_t)((registro >> n) & 0x01U);
```

Ejemplo práctico: Para comprobar la bandera de conversión lista (CNVR, en el bit 10) en el registro de alertas del sensor:

```
1 #define INA236_CNVR_Pos 10U
2 #define INA236_CNVR      (1U << INA236_CNVR_Pos)
3 uint16_t regValue = 0x0402; // Banderas del sensor (bit 10 en 1)
4
5 // Evaluacion con Alternativa A
6 if ((regValue & INA236_CNVR) != 0U) {
7     // La conversion esta lista (regValue & INA236_CNVR = 0x0400)
8 }
9
10 // Evaluacion con Alternativa B (retorna 1)
11 uint8_t cnvrState = (uint8_t)((regValue & INA236_CNVR) >> INA236_CNVR_Pos);
12 // cnvrState es igual a 1
```